Supplementary Material

Algorithms for High-Throughput FASTQ Preprocessing on Heterogeneous CPU–GPU Systems

Panchev, Gueorguiev, Gabrovska, Simeonov

Manuscript JCB-2026-0222.R1, revision R2

This file accompanies the manuscript. It contains two notes and two tables.

- **Note S1**: why NVIDIA Clara Parabricks could not be benchmarked on the evaluation node.
- **Note S2**: the GPU- and CPU-based FASTQ quality-control tools we examined, and how we checked each one.
- **Table S1**: wall times of the final-build sweep on the four WGS datasets (repeated runs, same-session baseline).
- **Table S2**: dated log of the optimisation attempts. The measurements in it come from the build of each date. They are not comparable with Table S1.

The ARM evaluation host is a single-socket Ampere Altra node (ARM Neoverse N1, 128 cores, 246 GiB RAM, 2× NVIDIA A100 80 GB PCIe, 2× Samsung PM9A3 NVMe in `md` RAID1). Source code, scripts, raw logs, and the summary of the final-build measurements are in the repository `https://github.com/d0bromir/fastp-gpu` under `docs/publication/supplementary/` and `benchmark_results/`. They are also archived at Zenodo (`https://doi.org/10.5281/zenodo.23303954`).

Note S1: NVIDIA Parabricks on ARM Neoverse-N1

S1.1 Purpose and result

A reviewer asked how FASTP-GPU compares with NVIDIA Clara Parabricks. We tried to run Parabricks on the evaluation node and could not. This note gives the evidence. It was collected on 16 August 2026.

All ten published `arm64` Parabricks images terminate with `SIGILL` on real input. The cause is an instruction-set mismatch. The `arm64` images are built for the NVIDIA Grace CPU (Neoverse-V2, ARMv9-A). The evaluation node has an Ampere Altra CPU (Neoverse-N1, ARMv8.2-A). The binaries use at least two feature sets that the node lacks: SVE (for example `cntb`, `whilelo`) and FEAT_LRCPC2 (for example `stlurb`).

The result is a statement about our hardware. It is not a statement about the performance or quality of Parabricks. We report no performance number for it.

S1.2 Test platform

CPU	ARM Neoverse-N1 (Ampere Altra), 128 cores, ARMv8.2-A
Absent features	no `sve`, no `ilrcpc` (FEAT_LRCPC2), no `i8mm`, no `bf16`, no `sha3`
GPU	2× NVIDIA A100 80 GB PCIe; driver 595.71.05, CUDA 13.2
OS	Ubuntu 25.10, Linux 6.17.0-41, glibc 2.42
Container runtime	Apptainer 1.4.5

S1.3 Image inventory

NVIDIA publishes 33 container tags for Parabricks in the NGC registry. We read the architecture of each from its manifest. 23 tags are `amd64` only. Three early tags are `arm64` only (`4.2.0-1.grace`, `4.3.1-1.grace`, `4.3.2-1.grace`). Seven tags are multi-arch (`4.4.0-1` to `4.7.1-1`). The tags with an `.ampere` suffix are `amd64` images. In those tags "Ampere" names the GPU generation, not the CPU. This gives ten `arm64` images.

S1.4 Result for every `arm64` image

Each image was run through the official entry point on a small BAM file: `pbrun bam2fq --in-bam example.bam --out-prefix out`, with the GPUs bound into the container (`apptainer exec --nv`). The trapping instruction was located by running the CPU-side `coverage` tool under `gdb`.

#	Tag	Fault address	Instruction	Symbol	Feature
1	4.2.0-1.grace	0xaaaaaaaac5228	`whilelo p0.s, xzr, x3`	`prepareAdapterSequences()+216`	SVE
2	4.3.1-1.grace	0xaaaaaaaac6328	`cntb x23`	`prepareAdapterSequences()+1992`	SVE
3	4.3.2-1.grace	0xaaaaaaaac6368	`cntb x23`	same	SVE
4	4.4.0-1	0xaaaaaaaac6368	`cntb x23`	same	SVE
5	4.5.0-1	0xaaaaaaaac6368	`cntb x23`	same	SVE
6	4.5.1-1	0xaaaaaaaac6368	`cntb x23`	same	SVE
7	4.6.0-1	0xaaaaaaaac6528	`cntb x23`	same	SVE
8	4.6.0-2	0xaaaaaaaac6528	`cntb x23`	same	SVE
9	4.7.0-1	0xaaaaaaaac65e8	`cntb x23`	same	SVE
10	4.7.1-1	0xaaaaaaaac65e8	`cntb x23`	same	SVE

A second trap site is in a different tool (`bam2fq`). It lies in `libfilehandle.so`, which every Parabricks tool links: `stlurb wzr, [x19, #28]` in `PBBamFile::getString(unsigned int)+420` (FEAT_LRCPC2). Registry digests of the tested images were recorded at test time

and are in the repository file `docs/publication/supplementary/parabricks-compatibility-evidence.md`.

Parabricks releases 4.5.0-1 and later print `Process terminated with signal [SIGILL: 4]`. Earlier releases abort at the same point and print only `Could not run bam2fq`. For those we called the binary directly and obtained exit code 132 (128 + SIGILL).

S1.5 Static instruction-set analysis

We disassembled every executable and shared library of image 4.5.1-1 with `objdump -d` and counted instructions that ARMv8.2-A does not implement. We matched mnemonics only, so the counts are lower bounds. The executables contain 4,431 SVE instruction sites in total. Seven of 26 executables are free of them (`dbsnp`, `deviceQuery`, `indexgvcf`, `pbsw`, `postpon`, `prepon`, `sort`). All seven are auxiliary utilities.

Shared library	SVE sites	LRCPC2 sites
`libbwalib.so`	213	0
`libfilehandle.so`	580	63
`libpairHMMlib.so`	311	0
`libswlib.so`	124	0

`libfilehandle.so` is linked into every tool. A tool whose own executable is clean, such as `bam2fq`, therefore still traps. The affected objects contain no `getauxval` or HWCAP references and no `IRELATIVE` relocations, so there is no runtime CPU dispatch.

S1.6 Controls

Alternative explanation	Test	Result
GPU driver broken	Repaired a driver/kernel-module mismatch, rebooted	`nvidia-smi` shows both A100s
GPU not visible in container	`apptainer exec --nv ... nvidia-smi`	Both GPUs visible
CUDA unusable	Parabricks `deviceQuery`	Runs, exit 0
Image corrupt	`pbrun version`, tool banners	Correct output
GPU code path at fault	`pbbwa --version`	Initialises the GPU, exit 0
Failure is GPU-related	CPU-only tools `coverage`, `collectmultiplemetrics`	Still `SIGILL`
Runtime dispatch misdetects	`objdump -T`, `readelf -r`	No HWCAP references, no IFUNC
One bad release	All ten images, 2023 to 2026	Same failure in all

S1.7 Minimal reproducer

A short C program that executes one SVE instruction (`ptrue p0.b`) on the node reports `HWCAP_SVE = NO` and ends with `Illegal instruction` (exit 132). To reproduce with an image:

```
apptainer pull --arch arm64 pb.sif \
    docker://nvcr.io/nvidia/clara/clara-parabricks:4.5.1-1
apptainer exec --nv pb.sif nvidia-smi
apptainer exec --nv -B /data:/indata -B /out:/outdata pb.sif \
    pbrun bam2fq --in-bam /indata/example.bam --out-prefix /outdata/o
```

The input was `example.bam` from the Falco test data. The failure does not depend on the input.

S1.8 Why there is no workaround

Parabricks is closed-source, so it cannot be rebuilt for ARMv8.2-A. The earliest `arm64` image (4.2.0-1.grace) already traps, and so does the newest (4.7.1-1). Binary translation could emulate SVE but cannot pass CUDA through to a physical GPU, and timings under emulation would be meaningless. The only other machine available to us is an x86-64 server without an NVIDIA GPU.

S1.9 Scope

The tools are also not comparable in function. The `pbrun` tool set of release 4.7.1-1 covers alignment, variant calling, and related secondary analysis. It has no standalone FASTQ quality-control, adapter-trimming, or filtering tool. `pbrun fq2bam` has no adapter-trimming or quality-filtering option. Its only filter option is a SAM-flag filter (`--filter-flag`). The tool list is in the Parabricks tool reference (`https://docs.nvidia.com/clara/parabricks/tool-reference`).

Note S2: GPU- and CPU-based FASTQ quality-control tools examined

S2.1 Purpose

This note records which tools we examined as possible comparators for FASTP-GPU, and what we could verify about each. We checked papers, package registries, and source repositories. The checks were made on 16 and 22 August 2026 and 24 August 2026 (RabbitQCPlus). The note makes no claim about why any tool is or is not maintained.

S2.2 Tools

Tool	Task	GPU code	Source obtainable	Used as comparator
FastQC	QC report only	No	Yes	Not in this paper
Falco	QC report only	No	Yes	Not in this paper
FASTP	QC, trim, filter	No	Yes	Yes (baseline)
RabbitQCPlus	QC, trim, filter	No	Yes (x86-64 only)	No
NVIDIA Parabricks	Alignment, variant calling	Yes	Container images only	No (Note S1)
G-CNV (Manconi et al., 2015)	Read filtering, masking, deduplication for CNV analysis	Yes (3 of 4 steps)	No: download link returns HTTP 404	No
CARE / CARE-GPU	Read error correction	Yes	Yes	No: different task
"Pre-processing of high-throughput sequencing data" (Manconi et al., 2017)	QC, filter, trim (as described)	Claimed	No: repository page returns HTTP 403	No
NGS-GPU pipeline (CIPF)	QC report, read filtering	Claimed	No: project host shows "Under maintenance"	No

S2.3 How each entry was checked

RabbitQCPlus. Its README and its peer-reviewed description (Yan et al., 2023) make no GPU claim. The repository has no `.cu` file. The `Makefile` sets `THRUST_DEVICE_SYSTEM=THRUST_DEVICE_SYSTEM_` for every build and never calls `nvcc`. It integrates the CARE error-correction engine, which has a separate GPU build. Eight core source files include x86-only SIMD headers (`immintrin.h`, `pmmintrin.h`) with no ARM fallback, so it did not build on the ARM node.

G-CNV. The paper (Frontiers in Bioengineering and Biotechnology, doi:10.3389/fbioe.2015.00028) states that G-CNV was designed for NVIDIA Kepler cards and CUDA 6.0 or later. The download address given in the paper returns HTTP 404. We found no mirror on GitHub or Software Heritage. Kepler (compute capability 3.5) is not supported by CUDA 12 and later.

2017 CNR item. A Google Scholar entry lists the title and authors and gives the CNR repository (`publications.cnr.it`) as the venue. We found no DOI. The repository handle redirects to a page that returns HTTP 403. We could not confirm that this item was peer-reviewed, so the manuscript does not cite it.

NGS-GPU pipeline. The project page (`docs.bioinfo.cipf.es/projects/ngs-gpu-pipeline`) shows a maintenance page. Wayback Machine captures exist for March 2012 to April 2016. The

release artefact named in archived pages is called `fastq-hpc-tools`. Downloads required a login. We never obtained the source, so we could not inspect it for device code.

CARE-GPU. The repository README requires CUDA 11 or newer and a Pascal or newer GPU. The A100 is Ampere (compute capability 8.0). The tool corrects sequencing errors. It does not perform quality control, trimming, or filtering.

Software Heritage and the Wayback Machine blocked automated retrieval. We did not pursue it further.

Table S1: Wall time of the final-build sweep, WGS datasets

Table S1: Final-build sweep on the ARM host (commit `8e87c0e`, release v1.3.3-fastp-d0bromir.17). Wall time is mean ± sample SD over n runs. CV is the coefficient of variation in percent. "Hung" is the number of runs of that cell that hung and were excluded from the mean. The baseline is FASTP v1.3.3 measured in the same session. Output was written to `/dev/shm`.

Dataset	T	Tool	n	Wall (s)	CV (%)	Hung
WGS PE (41.3 GB)	8	FASTP v1.3.3	5	691.1 ± 2.2	0.31	0
	8	FASTP-GPU CPU	5	447.8 ± 0.5	0.11	0
	8	FASTP-GPU GPU	5	389.3 ± 1.9	0.49	0
	16	FASTP v1.3.3	1	407.7	—	4
	16	FASTP-GPU CPU	5	241.0 ± 0.7	0.31	0
	16	FASTP-GPU GPU	5	241.9 ± 0.6	0.27	0
	32	FASTP v1.3.3	5	696.4 ± 6.5	0.93	0
	32	FASTP-GPU CPU	5	243.7 ± 0.5	0.21	0
	32	FASTP-GPU GPU	5	244.9 ± 1.9	0.76	0
WGS PE (19.4 GB)	8	FASTP v1.3.3	3	190.9 ± 0.2	0.11	0
	8	FASTP-GPU CPU	3	98.8 ± 0.1	0.10	0
	8	FASTP-GPU GPU	3	99.1 ± 0.2	0.18	0
	16	FASTP v1.3.3	1	161.0	—	2
	16	FASTP-GPU CPU	3	99.2 ± 0.1	0.12	0
	16	FASTP-GPU GPU	3	99.6 ± 0.2	0.19	0
	32	FASTP v1.3.3	3	168.1 ± 0.9	0.56	0
	32	FASTP-GPU CPU	3	99.6 ± 0.2	0.16	0
	32	FASTP-GPU GPU	3	100.1 ± 0.2	0.17	0
WGS PE (13.6 GB)	8	FASTP v1.3.3	3	132.9 ± 0.4	0.33	0
	8	FASTP-GPU CPU	3	71.3 ± 0.1	0.20	0
	8	FASTP-GPU GPU	3	70.5 ± 0.1	0.20	0
	16	FASTP v1.3.3	2	109.6 ± 0.8	0.73	1
	16	FASTP-GPU CPU	3	70.4 ± 0.2	0.27	0
	16	FASTP-GPU GPU	3	70.9 ± 0.2	0.35	0
	32	FASTP v1.3.3	3	115.2 ± 0.8	0.70	0
	32	FASTP-GPU CPU	3	71.1 ± 0.1	0.12	0
	32	FASTP-GPU GPU	3	71.5 ± 0.1	0.11	0
WGS SE (6.7 GB)	8	FASTP v1.3.3	0	—	—	3
	8	FASTP-GPU CPU	3	79.3 ± 0.1	0.11	0
	8	FASTP-GPU GPU	3	79.7 ± 0.0	0.05	0
	16	FASTP v1.3.3	5	195.4 ± 2.9	1.49	0
	16	FASTP-GPU CPU	5	79.3 ± 0.2	0.24	0
	16	FASTP-GPU GPU	5	79.6 ± 0.1	0.16	0
	32	FASTP v1.3.3	5	184.1 ± 2.0	1.07	0
	32	FASTP-GPU CPU	5	79.2 ± 0.2	0.23	0
	32	FASTP-GPU GPU	5	79.6 ± 0.2	0.21	0

Table S2: Optimisation attempt log

Table S2: Optimisation attempts. KEEP: the change is in the released binary. REJECT: reverted or never merged. PARK: deferred. Effects are wall-clock changes at $-w = 16$ unless stated.

Date	Attempt	Decision	Outcome
2026-05-15	GPU async initialisation overlap	KEEP	6.6% faster on a 1.2 GB panel at $-w = 8$. Hides the 3.9 s CUDA initialisation behind reader I/O.
2026-05-15	GPU init size gate (`FASTP_GPU_MIN_BYTES`)	KEEP	Opt-in. The default is 0 (gate off), so benchmarked runs always use the GPU.
2026-05-15	GPU gzip through a shared nvCOMP dispatcher	REJECT	One mutex serialises all writer streams. Wall time rises from 5.6% at $-w = 2$ to 130% at $-w = 16$.
2026-05-15	Runtime-configurable slot-pool size	PARK	Slot contention was not on the critical path. The knob (`FASTP_GPU_SLOTS`) stays in the code.
2026-05-16	ISA-L as BGZF DEFLATE backend	REJECT	+14% output size at matched throughput. Kept as an opt-in switch.
2026-05-16	Thread-fairness audit	REJECT	No code change. The high CPU figure that triggered it was the baseline's.
2026-05-17	BGZF device-side output pack with pinned caller buffer	KEEP	−473 ms dispatch time, as predicted.
2026-05-17	BGZF input scaling	KEEP (finding)	BGZF input is 1.59× faster than plain gzip at the best thread count (7.53 s against 11.94 s). No code change.
2026-05-17	nvCOMP multi-stream decompression	REJECT	Wall change within noise.
2026-05-17	Fused single packed write in the BGZF reader	KEEP	Wall time improves at every thread count.
2026-05-17	BGZF reader batch-size sweep	REJECT	The reader is faster but the wall floor is not set by it.
2026-05-17	BGZF reader split across both A100s	KEEP	`gpu_call_ms` drops 11 to 16%.
2026-05-17	BGZF thread-count sweep	KEEP (finding)	$-w = 16$ is the recommended setting for BGZF input on this host. No code change.
2026-05-17	Filter setup and slot-pool elimination	REJECT	Setup and teardown take 0.7 s. All three hypotheses were falsified.
2026-05-17	Filter dispatch audit under BGZF input	REJECT	Filter is balanced across both GPUs and takes 3% of worker wall time.
2026-05-17	`MAX_PACK_SIZE` sweep	REJECT	Smaller packs are monotonically worse at $-w = 16$ and 32.
2026-05-17	Worker-idle probe	KEEP	The probe is not in the source of the final build (commit `8e87c0e`).

Date	Attempt	Decision	Outcome
2026-05-18	Async producer thread for `fread`	REJECT	`io_read` is hidden but wall time is unchanged, because the compressor pool is saturated.
2026-05-18	BGZF output encode with nvCOMP DEFLATE	REJECT	2.2× slower than the CPU pool and 20% larger output at the usable quality.
2026-05-18	GPU k-mer contamination detection	KEEP	Off by default.
2026-05-18	`libdeflate` level 4 to 3	KEEP	+0.98% output size. 2 to 3% less user time. No wall-time effect shown.
2026-08-23	Grid sizing to the SM count for Algorithm 2	REJECT	+1.47% kernel time ($p < 0.01$, interleaved design).
2026-08-23	Sub-warp tiling with 128-bit loads for Algorithm 2	PARK	−0.61% kernel time, not significant (interleaved design). Kept on a development branch only.
2026-08-25	`FQ_BUF_SIZE` sweep	KEEP (finding)	16 MB. Smaller buffers are 5 to 8% slower on gzip input and 9 to 20% on BGZF input. Larger buffers show no gain.
2026-08-25	Compressor-thread sweep	KEEP (finding)	Near-linear to 4 threads, 4.66× at 8, flat from 8 to 32.